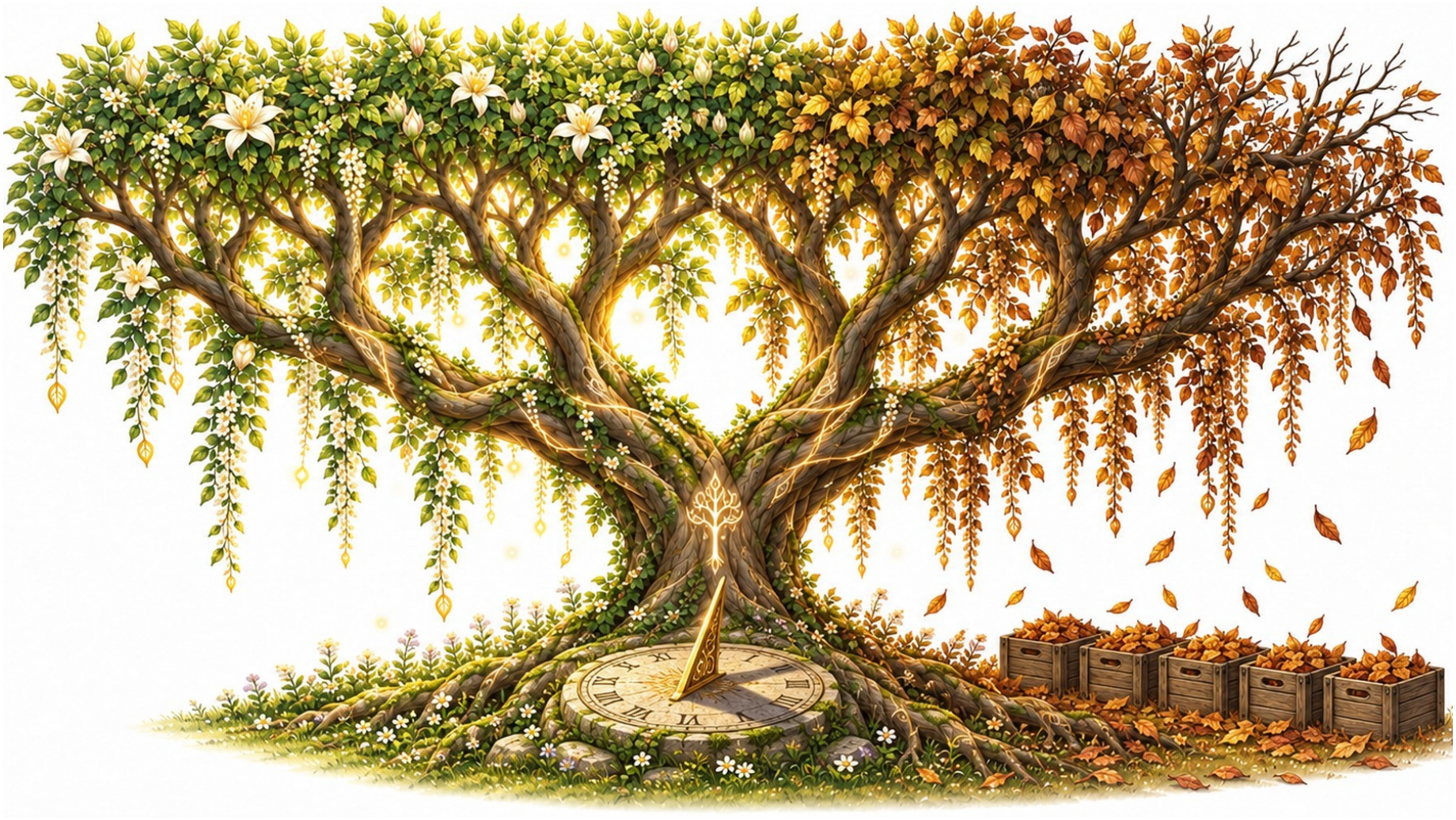
Resolver v1: 2.2× longer conflict history under the same memory

A temporal B+ tree for FoundationDB, with identical conflict decisions and a sequential first read path that is 2.7× slower, the target of the next milestone.

Beyond 5s, milestone 1 · Ricardo Jimenez-Peris · Inside DBs



6 GiB of history: **2.2× longer retention ▲** · **same decisions =** · **2.7× resolution time ▼**

By default, FoundationDB rejects a transaction when its read version has become more than five seconds old. The limit comes from how long the Resolvers retain the recent-write history used to check its conflicts: a fixed window, regardless of the memory available.

Resolver v1 keeps that history under a memory budget instead, and discards the oldest entries only when space runs out. This carousel gives the measurement first, then the representation, the memory layout, the bounded structural work and the reclamation that produce it.

1 What the measurement says

A 5,400-second trace of resolver work, 65,536 transactions per second with 16–64-byte keys, replayed through FoundationDB's skip list and through v1, both under 6 GiB of history. The skip list has no budget of its own, so its window was calibrated first: 265 seconds is the largest that fits; 270 exceeds the budget.

	FDB skip list	Resolver v1
History retained	265 s window	583 s minimum, 2.2× ▲
Resolution time, full trace	1,241 s	3,376 s, 2.7× ▼
Reads / writes	1× / 1×	5.1× / 2.7× ▼
Conflict decisions	same hash	same hash =
Reads rejected	none	none =

Under the same memory, v1 first discards history 703 seconds into the trace and never retains less than 583 seconds afterwards: 2.2 times the calibrated window, with identical decisions.

Resolution time goes the other way: 2.7 times the skip list's over the full trace, and 5.1 times on reads. This is a sequential first version that resolves each read and write individually, and every separator comparison reads key bytes outside the node, with nothing overlapping those accesses yet.

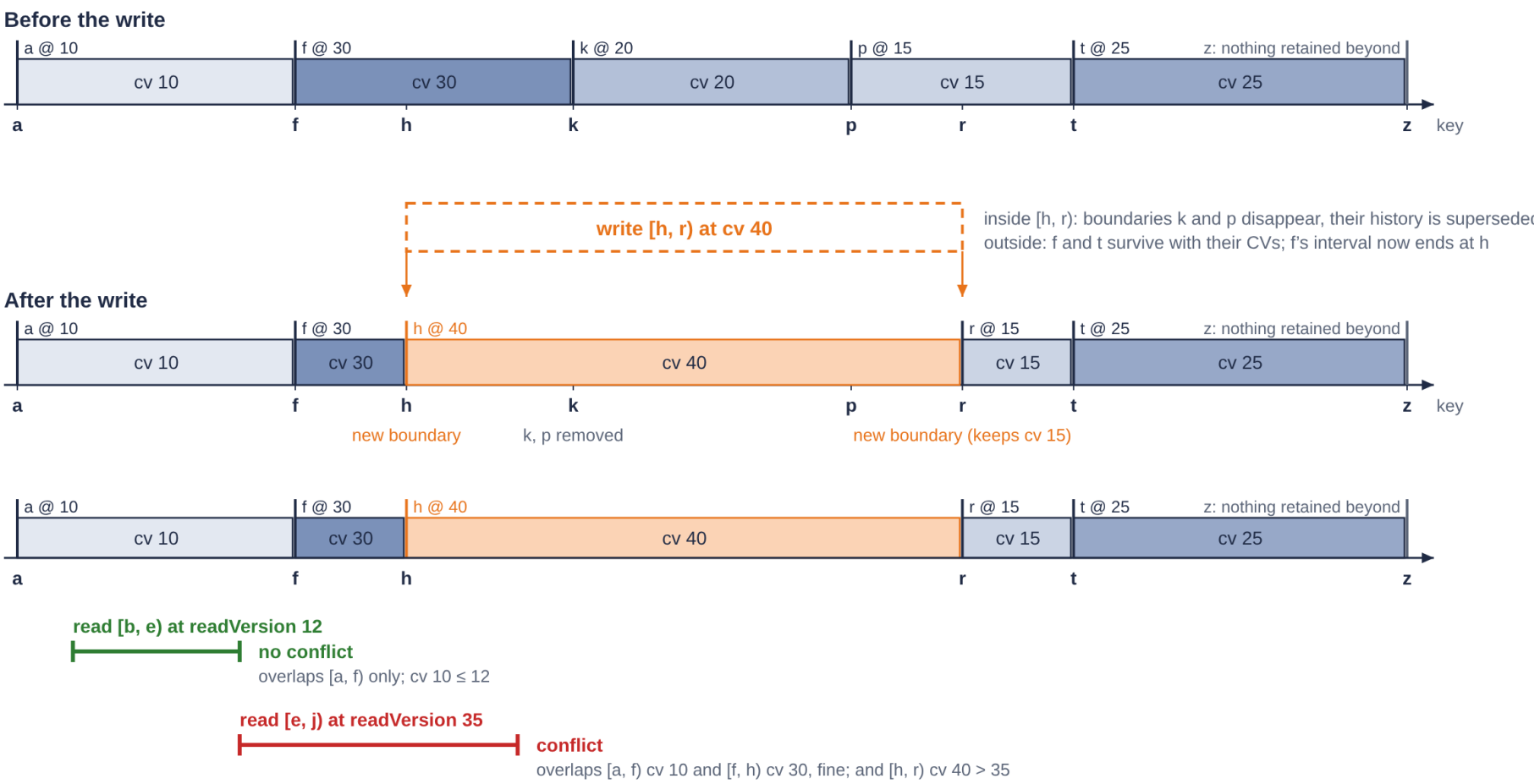
The result is a trade-off stated under equal memory: substantially more history for a higher cost per batch. Retention was this version's objective; resolution speed is the next one. ▲ better than the skip list, ▼ worse, = equal.

The next six pages show where the retention comes from and where the time goes.

2 What the resolver keeps

The history records which regions of the key space were written and at which commit version; it holds no values. A write at version 40 over [h, r) replaces the versions inside the range and keeps the exteriors: f's interval now ends at h, and [r, t) keeps its version 15. Intervals never overlap.

A read-conflict check is a range query: a transaction at read version RV conflicts if any interval overlapping one of its read ranges has a version greater than RV. The semantics are FoundationDB's; what v1 changes is the representation.



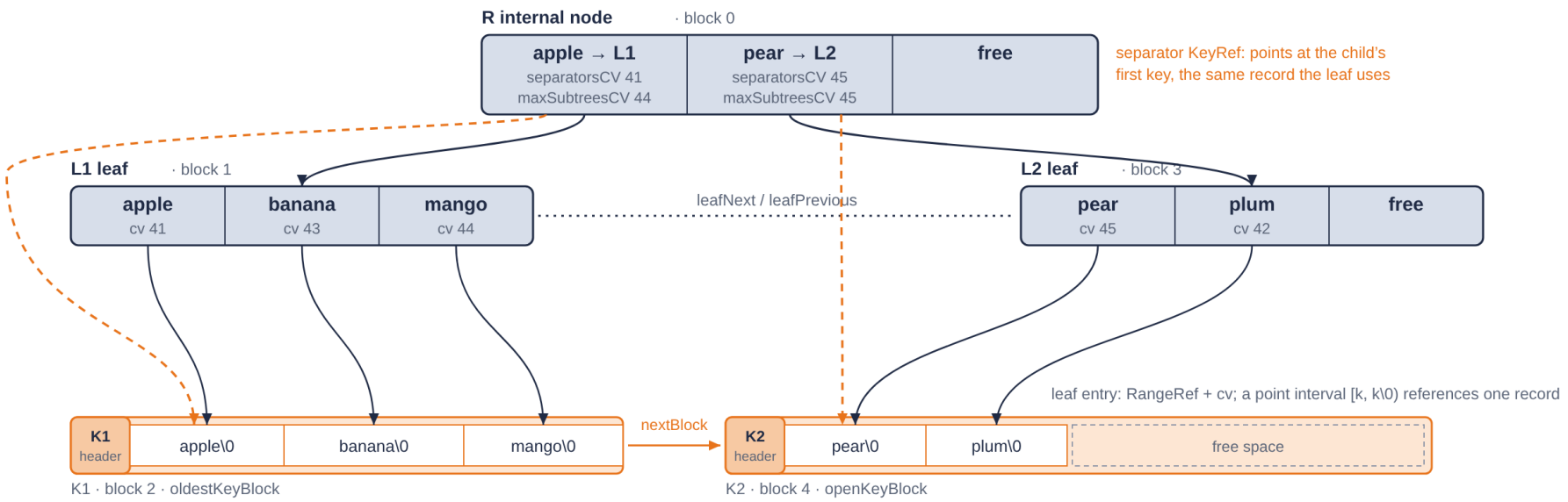
1 **`write [h, r) @ 40`**
k and p disappear, their history is superseded; f and t survive with their versions

2 **`read [e, j) @ RV 35`**
overlaps [h, r) with cv 40 > 35: conflict

3 **`read [b, e) @ RV 12`**
overlaps [a, f) only, cv 10 ≤ 12: no conflict

3 Where everything lives

Leaves hold intervals with their commit versions; internal nodes hold separators and child pointers. Key bytes never sit inside a node: a leaf entry references its key record, and a separator references the same record as the first entry of the child it routes to. Nodes and key blocks are blocks of the same pool.



1 2 KiB block = 40 children or 59 entries
node capacity does not depend on key length

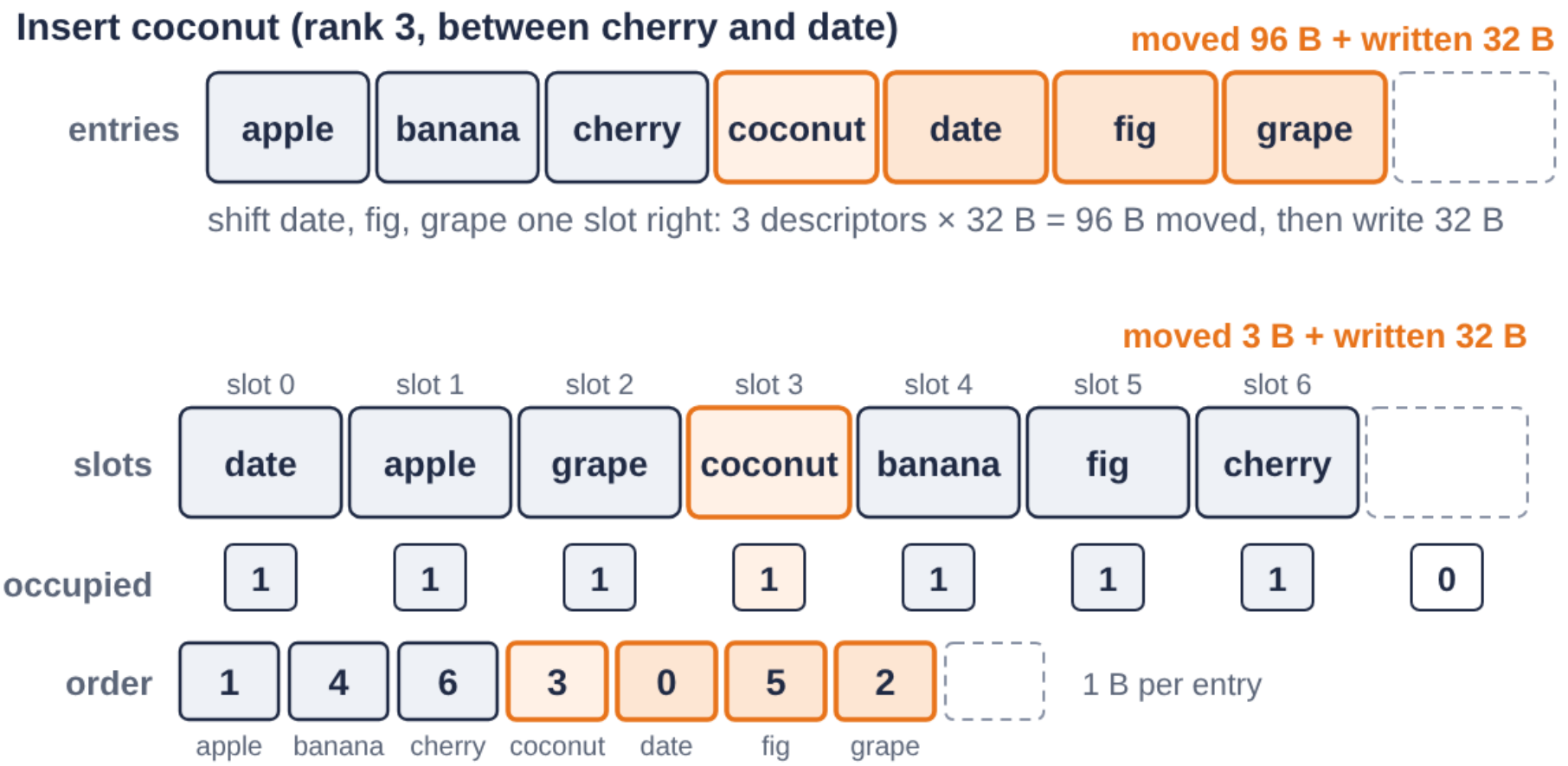
2 [k, k\0) stores k\0 once
a point write costs one record; the skip list keeps two copies of k

3 one pool for nodes and key blocks
a block released by a node can hold key records next, and the reverse

4 every separator compare reads outside the node
the price of 1–3, paid on every descent; v1 pays it sequentially

4 Keeping order without moving entries

Within a node, entries stay in the physical slot where they were written. A bitmap says which slots are occupied, and a one-byte permutation, order[], lists them in key order. Inserting writes the entry into a free slot and shifts one-byte indices; deleting clears a bit and shifts them back. Nothing else moves.



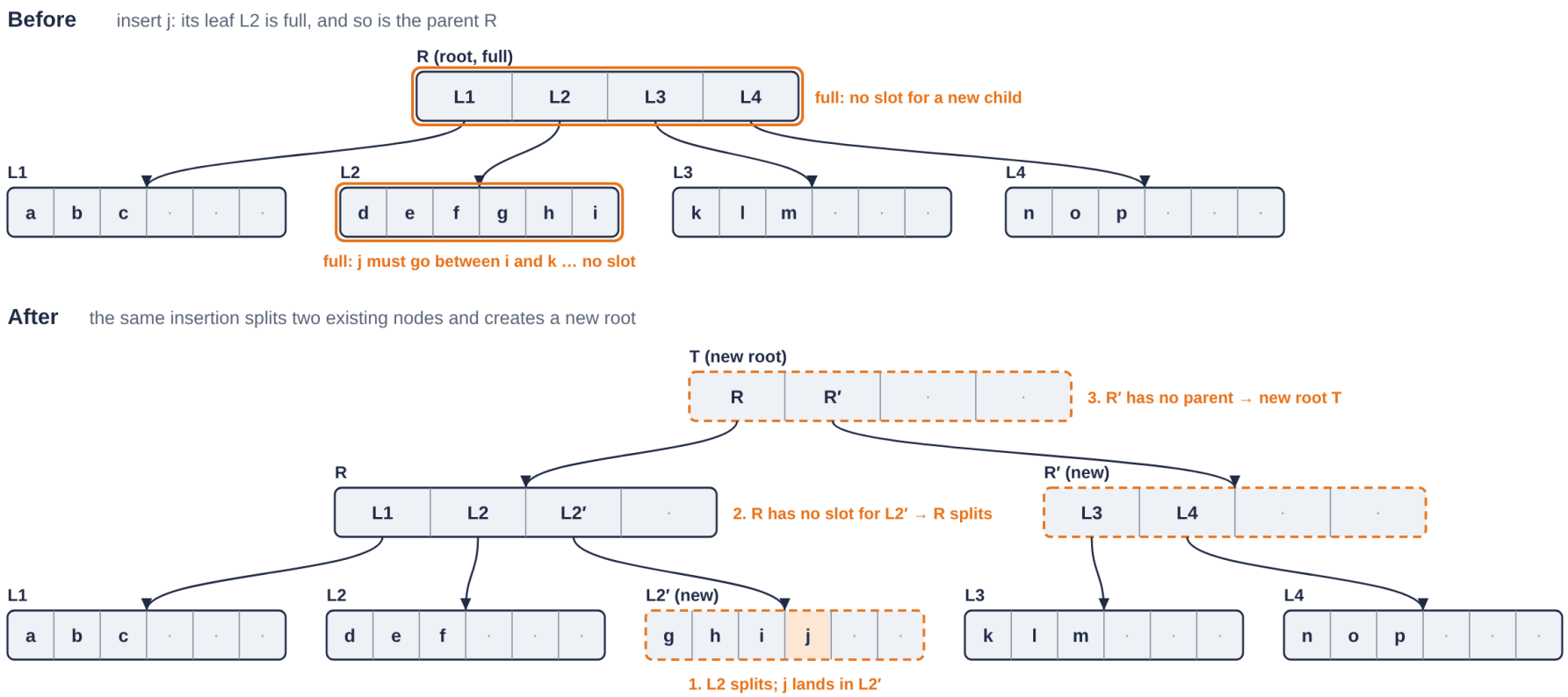
1 sorted array: move 3 descriptors, 96 B
then write the new one

2 v1: write into a free slot, shift 3 bytes
in a leaf of 59 entries, about 29 bytes instead of about 930

3 parentSlot is physical
a child's slot in its parent does not change when siblings come and go

5 Conventional splits cascade

A conventional B+ tree splits a node when it overflows. The new sibling needs a slot in the parent; if the parent is full it splits too, and the chain can run up to the root. The work of one insertion depends on how many of its ancestors happen to be full.



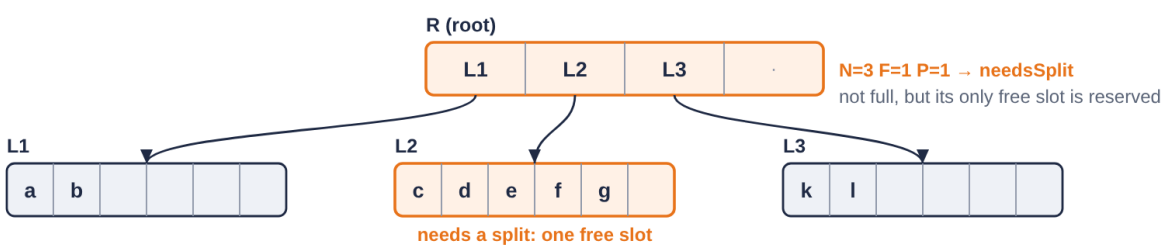
1 insert j into a full leaf
the leaf splits

2 the parent is full too
the root splits and a new root is created: two existing nodes split for one insertion

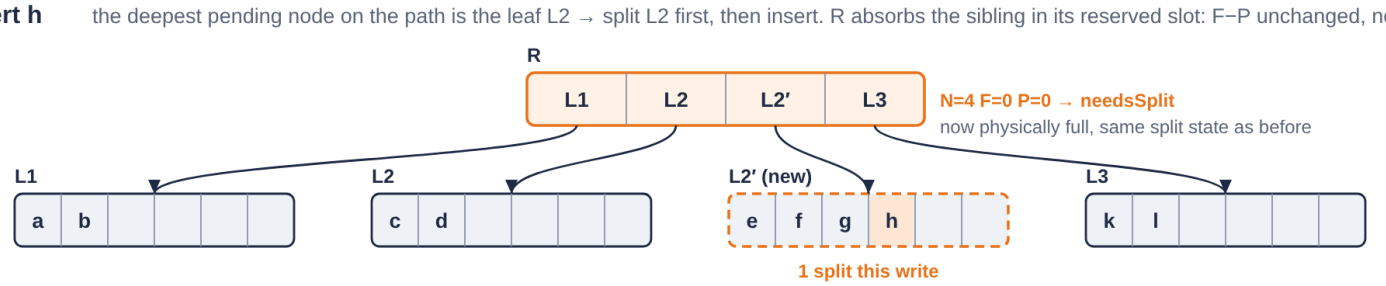
6 Splits that never cascade

V1 reserves capacity before it is needed. A leaf needs a split when it has at most one free slot, since a write can add two entries. Each internal node counts its children that need a split, P, and keeps one free slot for each; it needs a split itself when P equals its free slots F. A write splits the deepest node on its path that needs one, before inserting.

State 0 L2 has one free slot, so it needs a split; R reserves a slot for L2′



Insert h the deepest pending node on the path is the leaf L2 → split L2 first, then insert. R absorbs the sibling in its reserved slot: F−P unchanged, no cascade.



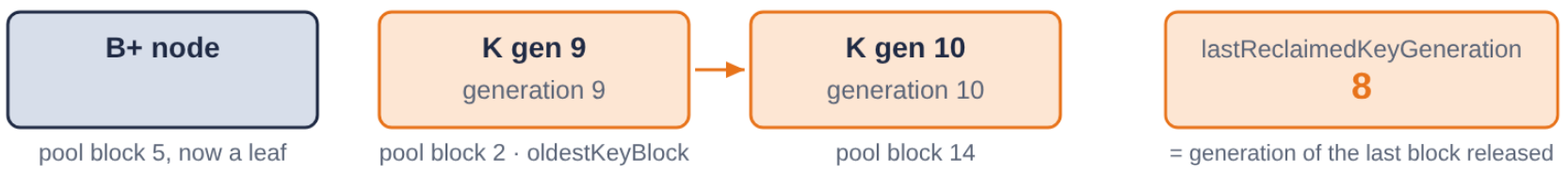
1 **L2 has one free slot: it needs a split**
R keeps a slot reserved for the sibling: N=3 F=1 P=1

2 **insert h: split L2 first, then insert**
R absorbs L2' in the reserved slot: F − 1, P − 1, F − P unchanged, no cascade

3 **at most one existing node splits per write**
a bound on structural work; copying keys and removing covered entries remain

7 Reclaiming history under a budget

When the pool runs out, reclamation compares the oldest key block with the oldest node, releases whichever is older and advances the cutoff reclaimCV; reads below it are rejected as too old, exactly as FoundationDB rejects reads older than its window. Releasing a block does not clean every reference to it, so each reference carries the generation of the block it names.



1 key blocks are released in allocation order
one counter, lastReclaimedKeyGeneration, bounds every released block

2 valid ⇔ generation > lastReclaimedKeyGeneration
one integer compare; the block is never read, even when its pool block was reused

3 stale entries are cleaned when a search meets them
expired separators are refreshed from their child; empty subtrees are detached as a unit

What comes next

The read path is where the gap is. The next milestone groups the read checks of a batch, keeps several descents in flight and prefetches the key bytes each one will compare, so that the latency of one external access overlaps useful work on others.

After the resolver, the series moves to the storage server, which must keep the data versions that a longer-lived transaction reads, and then to the protocol that coordinates both with the transaction's read version.

Which of these mechanisms would you have designed differently?

The article, with the complete figures
ricardojimenezperis.com/systems/fdb-resolver-v1

Design as implemented, decision record, journal, evaluation, evidence
github.com/ricardojimenezperis/fdb-beyond-5s

Code, tests and harness: FoundationDB fork, tag resolver-v1
github.com/ricardojimenezperis/foundationdb

Links also in the first comment

Inside DBs · Ricardo Jimenez-Peris
Notes on how databases actually work